

Catchment Analyzer

Pond siting and catchment estimation from contour maps

Phase 1 — Backend API

Indian Institute of Technology Bhilai | September 1, 2026

Repository https://github.com/devreddy1015/catchment_analyzer
API route POST <http://10.50.43.255:5249/api/analyzeContour>
Alias POST <http://10.50.43.255:5249/api/findCatchment>
Live docs <http://10.50.43.255:5249/docs>
Sample map contours_1m.kml

1 What it does

Upload a contour map as KML or KMZ. The service rebuilds the terrain surface, routes water across it, picks a pond site, and returns the catchment draining to that site as JSON.

Nothing is tied to the sample map. Extent, elevation range, contour interval, grid size and every normalisation range are read from whatever file arrives. The sample was used to test the code, not to tune it.

Requirement	Where
POST /analyzeContour or /findCatchment	Both; the second is an alias (§5)
Accept contour map as upload	multipart/form-data, KML + KMZ
Analyse terrain	§3, stages 1–4
Identify pond location	§3, stage 6
Estimate catchment area	§3, stage 7
Structured JSON	§5
Demonstrate on sample map	§6
No hard-coded results	§4

2 Architecture

FastAPI handles HTTP and nothing else; the analysis core is plain Python with no web imports, so it can also be driven from a script or a batch job.

Module	LOC	Job
core/kml.py	295	Parse KML/KMZ; derive interval and bounds
core/grid.py	183	Interpolate contours onto a square-cell grid
core/hydrology.py	408	Sink fill, D8 routing, delineation
core/siting.py	518	Scoring, storage, ranking
core/watercourse.py	340	River / floodplain / nala classification
core/pipeline.py	533	Orchestration and response assembly
api/routes.py	209	Endpoints and validation

Table 1: About 3500 lines in total. One stage per module.

Pipeline. parse → interpolate → fill sinks → D8 route → classify watercourses → score sites → delineate catchment → quantify yield.

3 How the catchment is found

This is the standard raster-hydrology chain, built on published algorithms rather than invented rules. Pseudocode below; the code is in the repository.

3.1 Stage 1 — Read the contours

A KMZ is a zip; the .kml inside is extracted first. Elevation comes from whichever place the file actually puts it, since files in the wild differ.

Algorithm 1 — Read contours

```

1 function read_contours(bytes):
2   if bytes is a zip then bytes <- extract largest .kml member
3   foreach placemark in parse_xml(bytes):
4     coords <- parse_coordinates(placemark)
5     z <- first available of: coordinate altitude,
6                               <ExtendedData> elevation,
7                               numeric <name>
8     if z exists and coords has >= 2 points then keep ContourLine(coords, z)
9
10  drop levels isolated from the series    // spot heights, stray labels
11  assert >= 3 lines and >= 2 distinct levels
12
13  bounds <- min/max of all vertices      // all DERIVED, never assumed
14  interval <- median gap between levels
15  return ContourSet(lines, bounds, levels, interval)

```

3.2 Stage 2 — Build a surface

Contours are scattered points carrying a z value. A Delaunay-based linear interpolation turns them into a grid, with nearest-neighbour fill outside the hull and light smoothing to remove triangulation facets.

Algorithm 2 — Interpolate onto a grid

```

1 function build_grid(contours, resolution = 160):
2   (s, w, n, e) <- contours.bounds, padded slightly
3
4   // Cells are kept SQUARE IN METRES -- D8 assumes it.
5   if width_m >= height_m then
6     cols <- resolution; rows <- resolution * height_m / width_m
7   else
8     rows <- resolution; cols <- resolution * width_m / height_m
9
10  targets <- regular lon/lat mesh (rows x cols), north-up
11  values <- LinearNDInterpolator(vertices, elevations)(targets)
12  values[NaN] <- NearestNDInterpolator(...)    // outside the hull
13  surface <- uniform_filter(values)
14
15  return ElevationGrid(surface, bounds, cell_size_m, interval)

```

3.3 Stage 3 — Fill the sinks

Routing on a raw surface strands water in every hollow. Priority-flood filling raises each pit to its spill level. The fill depth is not waste: it *is* the depth of the hollow, which stage 6 uses as evidence of natural storage.

Algorithm 3 — Priority-flood sink fill

```

1 function fill_sinks(z, epsilon = 0):
2   filled <- copy of z
3   queue <- min-heap of ALL BOUNDARY cells, keyed by elevation
4
5   while queue not empty:
6     (level, r, c) <- pop lowest
7     foreach unvisited 8-neighbour (nr, nc):
8       mark visited
9       // no cell can sit lower than the lowest rim reaching it
10      filled[nr, nc] <- max(filled[nr, nc], level + epsilon)
11      push (filled[nr, nc], nr, nc)
12   return filled
13
14 depression_depth(z) = max(0, fill_sinks(z, 0) - z)
15 conditioned_surface(z) = fill_sinks(z, 1e-4) // tilt flats so flow is defined

```

The ϵ tilt matters: a filled pit is a flat lake where no cell has a downhill neighbour, so flow would be undefined across it.

3.4 Stage 4 — Route the water**Algorithm 4 — D8 direction and accumulation**

```

1 function flow_direction(z, cell_size_m):
2   // each cell drains to its steepest downhill neighbour of eight
3   padded <- z padded with +infinity // off-grid can never win
4   foreach k, (dr, dc) in the 8 offsets:
5     slope <- (z - padded shifted by (dr,dc)) / (cell_size_m * dist[k])
6     direction <- where(slope > best, k, direction)
7   return direction // -1 marks a pit
8
9 function flow_accumulation(z, direction):
10  // water only moves downhill, so highest-to-lowest means a cell's
11  // total is final before it is passed on -- one linear pass
12  accumulation <- ones
13  foreach cell sorted by z descending:
14    d <- downstream neighbour
15    if d exists then accumulation[d] += accumulation[cell]
16  return accumulation

```

3.5 Stage 5 — What the ground already is

Before proposing anything, the map is split by *upstream area*, so the rule transfers to other terrain unchanged.

Class	Test	Result
Pond ground	upstream $\leq$ 25 ha	pond proposed
Nala	25 < upstream $\leq$ 100 ha	bund / percolation tank
River	upstream > 100 ha	nothing proposed
Floodplain	within 40 m, HAND < 3 m	nothing proposed
Still water	flat pooled area $\geq$ 2 ha	nothing proposed

Past roughly 25 ha a storm delivers more than a farm pond can hold, and the problem becomes a spillway rather than an embankment. Reporting that as a different structure is more useful than silently recommending the wrong one.

3.6 Stage 6 — Choose the site

Four layers, each normalised against the range present *in the uploaded map* — so the score is relative to that terrain, never to a fixed constant.

$$n(x) = \frac{x - \min x}{\max x - \min x}, \quad S = 100 \cdot \frac{\sum_i w_i L_i}{\sum_i w_i} \quad (1)$$

Layer	Value	Weight
Depression	$n(\text{fill depth})$	0.30
Catchment	$n(\log(1 + A))$	0.30
Slope	$n(1/(1 + \theta/8^\circ))$	0.25
Elevation	$n(-z)$	0.15

The logarithm on accumulation matters — raw upstream counts span orders of magnitude, and without it one trunk channel saturates the layer and flattens every real difference elsewhere.

Algorithm 5 — Score, filter, rank

```

1 function select_sites(grid, accumulation, depths, max_sites):
2   score <- 100 * weighted_sum(four normalised layers) / sum(weights)
3
4   blocked <- river or floodplain or still_water or outer edge ring
5   // busy channel with no hollow the source could measure = interpolation
6   holds <- max(0.30, grid.vertical_resolution_m)
7   blocked <- blocked or (accumulation >= percentile(accumulation, 95)
8                     and depths < holds)
9
10  foreach cell in unblocked cells sorted by score descending:
11    if enough sites then break
12    if too close to an accepted site then continue
13
14    storage <- natural_storage(grid, depths, cell)
15    inflow <- upstream_area * 0.4 * 50mm // one ordinary storm
16    if inflow > storage.volume then
17      reclassify as channel structure (bund + waste weir)
18    else
19      accept as PondSite

```

Storage is measured, not assumed from a design depth. The sink fill already gives the spill elevation, so the pond is grown out to that real rim:

Algorithm 6 — Storage by flood fill to the rim

```

1 function natural_storage(grid, depths, cell):
2   if depths[cell] <= 0 then return zero // no hollow; must be dug
3   spill <- grid.z[cell] + depths[cell]
4
5   flood outwards from cell over neighbours with z < spill:
6     head_sum <- head_sum + (spill - z of each flooded cell)
7
8   return { depth, spill, area = count * cell_area,
9           volume = head_sum * cell_area }

```

3.7 Stage 7 — Delineate the catchment

The catchment is exactly the set of cells whose flow pointers lead to the site — found by walking the D8 network upstream from it.

Algorithm 7 — Delineate

```

1 function delineate(grid, direction, outlet):
2   contributors <- invert(direction)      // downstream -> list of upstream
3   cells <- {outlet}; queue <- [outlet]
4   while queue not empty:
5     foreach up in contributors[pop queue] not already in cells:
6       add up to cells and queue
7
8   area      <- count(cells) * cell_area
9   polygon   <- union of cell squares
10  flow_path <- longest upstream chain, in metres
11  gradient  <- (max_z - min_z) / flow_path
12  return Catchment(area, polygon, flow_path, gradient, relief, slope)

```

3.8 Stage 8 — Yield

$$V = P \cdot A \cdot C \quad t_c = 0.0195 L^{0.77} S^{-0.385} \quad (2)$$

P rainfall (mm), A area (m²), C runoff coefficient (default 0.4), L longest flow path (m), S gradient. With no rainfall given, the response reports yield per millimetre so local rainfall can be applied later.

4 Nothing is hard-coded

Quantity	Comes from	On the sample
Geographic extent	vertex min/max	21.240–21.264 N
Elevation range	contour z values	267–298 m
Contour interval	median level gap	1.0 m
Grid shape	extent aspect ratio	131 × 160
Cell size	haversine / cols	20.6 m
Score normalisation	the map's own range	per upload
Pond location	argmax of score	computed
Catchment area	upstream cell count	computed

The only constants are physical ones that hold anywhere: the 25 ha and 100 ha thresholds, the 8° slope half-score, the 50 mm storm test and Kirpich's coefficients. None encodes a location. A second endpoint, `/analyzeArea`, runs the same core on terrain downloaded for arbitrary world coordinates — reasonable evidence the analysis is not tuned to one map.

5 API

Method	Path	Purpose
POST	<code>/api/analyzeContour</code>	Analyse an uploaded contour map
POST	<code>/api/findCatchment</code>	Alias of the above
POST	<code>/api/analyzeArea</code>	Analyse by coordinates, no file
GET	<code>/api/health</code>	Liveness check
GET	<code>/docs</code>	Interactive documentation

The analysis route is POST-only

It carries a file in the request body, so it cannot be opened from a browser address bar — a GET correctly returns 405 Method Not Allowed. To try it interactively, open <http://10.50.43.255:5249/docs>, which gives an upload form wired to the live endpoint.

5.1 Request — multipart/form-data

Field	Type	Default	Meaning
file	file	required	.kml or .kmz, ≤ 64 MB
resolution	int	160	Cells along the longer side (48–260)
max_sites	int	5	Candidate sites returned
runoff_coefficient	float	0.4	C in the Rational method
rainfall_mm	float	null	If given, absolute runoff volume

5.2 Response — 200 OK, abridged

```
{
  "success": true,
  "analysis_id": "ca_aff346d081",
  "source": { "format", "contour_lines", "vertices",
              "contour_interval_m", "bounds" },
  "terrain": { "min_elevation_m", "max_elevation_m", "relief_m",
               "mean_slope_deg", "mapped_area_km2", "grid" },
  "pond_site": { "latitude", "longitude", "elevation_m", "slope_deg",
                 "depression_depth_m", "upstream_hectares", "structure",
                 "score", "rating", "score_breakdown", "storage" },
  "catchment": { "outlet", "area_m2", "area_hectares", "cell_count",
                  "perimeter_m", "longest_flow_path_m", "average_gradient",
                  "time_of_concentration_min", "runoff" },
  "alternative_sites": [...], "channel_structures": [...],
  "watercourses": {...}, "overlays": {...},
  "geojson": { "type": "FeatureCollection", "features": [...] },
  "warnings": [...]
}
```

5.3 Errors

All share the shape {success, error, detail}.

400	Not a KML/KMZ, or unreadable archive
405	GET on a POST-only route
413	Upload over 64 MB
422	Fewer than 3 contour lines or 2 distinct levels
500	Interpolation produced an incomplete surface

6 Demonstration

```
curl -X POST \apiroute -F "file=@contours_1m.kml"
```

Returned HTTP 200, 43,588 bytes, in about 0.5 s.

Read from the file		Reconstructed terrain	
Contour lines	1,355	Grid	131×160
Vertices	159,113	Cell size	20.605 m
Distinct levels	32	Mapped area	8.90 km ²
Contour interval	1.0 m	Relief	29.13 m
Elevation range	267–298 m	Mean slope	2.37°

Pond site (rank 1)		Its catchment	
Location	21.249651, 81.286503	Area	9.47 ha
Structure	Farm pond	Cells	223
Score	54.3 (Moderate)	Perimeter	1,867.6 m
Elevation	277.38 m	Relief	9.61 m
Slope	1.11°	Flow path	601.4 m
Depression	2.74 m	Gradient	0.01599
Spill elevation	280.12 m	t_c	13.2 min
Water surface	21,228 m ²	Yield	37.87 m ³ /mm
Storage	26,811 m³	Share of map	1.06 %

6.1 The numbers check out

Both headline figures reproduce from the formulae in §3:

$$\text{yield} = \frac{94678.2 \times 0.4}{1000} = 37.87 \text{ m}^3/\text{mm} \quad \checkmark \quad (3)$$

$$t_c = 0.0195 \times 601.4^{0.77} \times 0.01599^{-0.385} = 13.2 \text{ min} \quad \checkmark \quad (4)$$

6.2 Ground it refused

River 51.46 ha, floodplain 229.18 ha, nala 9.38 ha, truncated channels 1.70 ha — **32.6 % of the map carries no pond proposal**. Where a pond is wrong the service names the structure that is right: the best alternative here is a nala bund at 21.247414, 81.289911 serving 51.46 ha, scoring 65.5 (*Good*) — higher than the pond, correctly reported as something else.

Figure

Save the map screenshot as `report/figures/catchment-map.png` and recompile — the figure appears here automatically.

7 Extending it, and what it cannot do

Built to extend. `build_grid` is shared by the contour path and the DEM path, so a grid from a raster has the same geometry as one from contours and every later stage is source-agnostic — adding GeoTIFF input means writing one reader. Thresholds are in hectares and metres, so no retuning for a new district. Resolution is a request parameter. The core imports nothing from FastAPI.

Limits, stated plainly. Interpolation cannot recover features finer than the contour interval, so hollows under 1 m are indistinguishable from smoothing artefacts on this map. About 7.6 % of the grid lies outside the contour hull and is extrapolated, which is why the outer ring is excluded from siting. Catchments are clipped by the map extent — flow from beyond it is not counted, and the response warns when a catchment touches the boundary. The Rational method suits small catchments, not basin-scale routing.

8 Running it

```
git clone \repourlraw && cd catchment_analyzer
python3 -m venv .venv && .venv/bin/pip install -r backend/requirements.txt
.venv/bin/uvicorn backend.main:app --host 0.0.0.0 --port 5249

curl -X POST http://localhost:5249/api/analyzeContour -F "file=@contours_1m.kml"
```

Or `docker compose up --build`, which publishes on port 5249.

References

- [1] O’Callaghan, J.F. & Mark, D.M. (1984). The extraction of drainage networks from digital elevation data. *CVGIP*, 28(3), 323–344.
- [2] Wang, L. & Liu, H. (2006). An efficient method for identifying and filling surface depressions in DEMs. *IJGIS*, 20(2), 193–213.
- [3] Barnes, R., Lehman, C. & Mulla, D. (2014). Priority-flood. *Computers & Geosciences*, 62, 117–127.
- [4] Kirpich, Z.P. (1940). Time of concentration of small agricultural watersheds. *Civil Engineering*, 10(6), 362.